

OCL2PSQL: An OCL-to-SQL Code-Generator for Model-Driven Engineering

Hoàng Nguyễn Phước Bảo and Manuel Clavel

Vietnamese-German University

27 November 2019, Nha Trang, Việt Nam

Background

Model-driven engineering (MDE)

- Model-driven engineering (MDE) is a software development *methodology* that focuses on create *models* of different system views.
- MDE success heavily depends on the *automatic* generated system *artifacts*, such as codes or configuration files.
- MDE had been guilty of making overambitious claims, but recently it found one of the domain that can deliver its full potential: *Security*.

Background

Object Constraint Language (OCL)

- Object Constraint Language (OCL) is a language for specifying constraints and queries using a textual notation. It is part of the UML standard.
- OCL expressions are written in the context of a model. called the *contextual* model.
- OCL is strongly typed. Expressions either have a *primitive* type, a *class* type, a *tuple* type, a *collection* type, or a *special* type.
- OCL is called to play a key role in model-driven engineering.

Background

Structure Query Language (SQL)

- Structure Query Language (SQL) is a language for managing *data* in relational database management systems.
- SQL scope includes data insert, query, update, delete and schema creation and modification.
- SQL is a declarative language, it also supports procedural elements.
- SQL systems provide *access control* support for *role-based* but none of them provide any *fine-grained* mechanism.

Motivation

OCL code-generators

- Propose a *model-driven methodology* for enforcing *fine-grained* access control policies on *database* applications.
- This is the case for the development of secure database-centric applications [M. de Dios, C. Dania, D. A. Basin, M. Clavel, 2014].
 - ▶ *Integrity constraints* and *authorization constraints* (fine-grained access control policies) can be specified using OCL.
- To this end, appropriate *code-generators* must exist to bridge the gap between UML/OCL, security policy *models* and SQL secure-enforced code.

Overview

OCL-to-SQL code-generators

- Different mappings from OCL to SQL have been proposed in the past, each with its own goals and limitations.
- There are two groups of challenges for OCL-to-SQL mappings:
 - ▶ *language coverage*, i.e., how much part of the OCL language a mapping can cover.
 - ▶ *execution-time efficiency*, i.e., how much time it takes to execute a query generated by a mapping.

OCL-to-SQL Mappings

Previous proposals

- **OCL2SQL** [F. Heidenreich, C. Wende, B. Demuth, 2008]. It generates SQL queries from OCL expressions, based on a set of transformation *templates*.
 - ▶ It only covers (a subset) of OCL boolean expressions.
 - ▶ Generated queries are very inefficient, even for medium-size scenarios (more than 45 minutes for relatively simple queries).
- **MySQL4OCL** [M. Egea, C. Dania, and M. Clavel. 2010]. It generates SQL queries from OCL expressions using imperative features of SQL: namely, variables, cursors and loops inside stored procedures. It also uses temporary tables to store intermediate results.
 - ▶ It covers a larger subclass of OCL expressions (including nested iterators).
 - ▶ It cannot benefit from the highly-optimized strategies implemented by SQL engines.
- **SQL-PL4OCL** [M. Egea, C. Dania, 2017]. It closely follows the design of MySQL4OCL and bears the same limitations.

OCL-to-SQL Mappings

Our current proposal

- **OCL2PSQL** [H. Nguyễn, M. Clavel, 2019]. It generates SQL queries from OCL expressions using *pure* SQL.
 - ▶ It covers a large subclass of OCL expressions (including nested iterators).
 - ▶ for intermediate results, it uses standard subqueries.
 - ▶ for iterator expressions, it adds to the subquery corresponding to the iterators' body an extra column corresponding to the iterator's variable. Intuitively, for each row, this column stores the element in the iterator's source that is "responsible" for the value stored in that row.

Prototype available at:

<http://cs.vgu.edu.vn/se/tools/ocl2psql/>

Source code available at:

<https://github.com/ocl2psql/OCL2PSQL>

OCL to Pure SQL (OCL2PSQL)



Vietnamese - German University

For me on GitHub

This web service is intended for readers of our manuscripts:

- Mapping OCL into SQL: Challenges and Opportunities Ahead
- OCL2PSQL: An OCL-to-SQL Code-Generator for Model-Driven Engineering

In particular, the *contextual model* of the OCL expressions to be translated is the model CarOwnership ([See here](#)).
The generated SQL expressions are only intended for the database schema CarDB ([See here](#)).

Please, be aware that OCL2PSQL is an on-going research project.
In particular, OCL2PSQL does not cover yet the full OCL language.

Moreover, for the time being, OCL expressions should adjust to the following syntactical conventions:

- Use single quotation marks for string-literals. E.g., `p.Person:name = 'string'`.
- **allInstances-expressions.** `<class>.allInstances()` should be written `<class>::allInstances()`. E.g., `Car::allInstances()`.
- **dot-expressions.** `<var>.<attr>` should be written `<var>.<class>:<attr>`, where `<class>` is the class of `<attr>`. E.g., `p.Person:name`.
- Similarly, `<var>.<assoc-end>` should be written `<var>.<class>:<assoc-end>`, where `<class>` is the source-class of `<assoc-end>`. E.g., `c.Car:owners`.

```
Car::allInstances()->collect(c|c.Car:color)
```

Translate

☒ Description Mode ?

Copy

```
/** BEGIN: Car::allInstances()->collect(c|c.Car:color) */  
SELECT TEMP_body.val AS val, TEMP_body.res AS res  
FROM (   
  /** BEGIN: c.Car:color */  
  SELECT 1 AS val, color AS res, Car_id AS ref_c  
  FROM Car  
  /** END: c.Car:color */  
 ) AS TEMP_body  
/** END: Car::allInstances()->collect(c|c.Car:color) */
```

OCL2PSQL

Language coverage

- Class operations

- ▶ `allInstances` `class.allInstances()`
- ▶ `attributes` `var.att`
- ▶ `association ends` `var.assoc`

- Constants operations

- ▶ `Boolean` (`TRUE`, `FALSE`) `Integer`(..., -1, 0, 1, ...) `String`('FDSE').

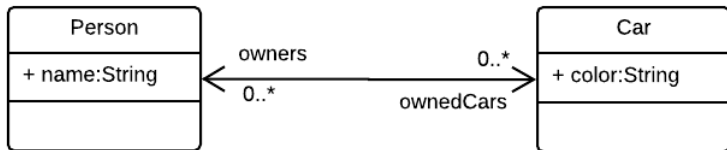
- Boolean operations

- ▶ `Logical operations` `NOT`, `AND`, `OR`, `XOR`
- ▶ `Comparison` `=`, `>`, `<`, `≤`, `≥`, `<>`
- ▶ `Generalization` `isKindOf(class)`, `isTypeOf(class)`
- ▶ `Undefinedness` `oclIsUndefined()`

- Iterative operations

- ▶ `collect` `source` → `collect(var|mapping body)`
- ▶ `forAll` `source` → `forAll(var|condition body)`
- ▶ `exists` `source` → `exists(var|condition body)`
- ▶ `select` `source` → `select(var|condition body)`
- ▶ ...

Example: UML data model



- There are *cars* and *persons*.
- Persons can own cars (they are their *owners*), and cars can be owned by persons (they are their *ownedCars*).
- No restriction is imposed regarding *ownership*: a person can own many different cars (or none), and a car can be owned by many different persons (or by none).
- Each car can have a *color*, and each person can have a *name*.

Example: SQL implementation

- A table `Car` to store the cars, with a column `color` to store the color of each car, and a column `Car_id` to store the (primary) *key* of each car.
- A table `Person` to store the persons, with a column `name` to store the name of each person, and a column `Person_id` to store the (primary) *key* of each person.
- A table `Ownership` to store the many-to-many relationship of ownership between cars and persons, with columns `ownedCars` and `owners` storing the (foreign) *keys* of the corresponding cars and persons.

Example: OCL queries

Evaluate language coverage

- *Which cars are in the system?*

```
Car.allInstances()
```

- *Which car's colors are in the system?*

```
Car.allInstances()->collect(c|c.color)->asSet()
```

- *How many cars have color 'no-color'?*

```
Car.allInstances()->select(c|c.color = 'no-color')->size()
```

- *Is there at least one car whose color is different from 'no-color'?*

```
Car.allInstances()->exists(c|c.color <> 'no-color')
```

- *How many cars have at least one owner whose name is 'no-name'?*

```
Car.allInstances()->select(c|c.owners  
->exists(p|p.name = 'no-name'))->size()
```

- Car.allInstances()

```
SELECT 1 AS val, Car_id AS res  
FROM Car;
```

- Car.allInstances()->collect(c|c.color)->asSet()

```
SELECT DISTINCT TEMP_src.val AS val, TEMP_src.res AS res  
FROM  
  (SELECT TEMP_body.val AS val, TEMP_body.res AS res  
    FROM  
      (SELECT 1 AS val, color AS res, Car_id AS ref_c  
        FROM Car  
      ) AS TEMP_body  
    ) AS TEMP_src;
```

- `Car.allInstances()->select(c|c.color = 'no-color')->size()`

```
SELECT 1 AS val, COUNT(*) AS res FROM
  (SELECT TEMP_gbody.val AS val, TEMP_gbody.ref_c AS res
  FROM
    (SELECT * FROM
      (SELECT CASE WHEN TEMP_LEFT.val = 0
      OR TEMP_RIGHT.val = 0 THEN 0 ELSE 1 END AS val,
      TEMP_LEFT.res = TEMP_RIGHT.res AS res,
      TEMP_LEFT.ref_c AS ref_c
    FROM
      (SELECT 1 AS val, color AS res, Car_id AS ref_c
      FROM Car
    )AS TEMP_LEFT JOIN
    (SELECT 1 AS val, 'no-color' AS res) AS TEMP_RIGHT
  ) AS TEMP_body
  WHERE TEMP_body.res = 1
  ) AS TEMP_gbody
) AS TEMP_src;
```

Example: Scenarios

Evaluate execution-time efficiency

- $\text{CarDB}(n)$ denote an instance of the database containing 10^n cars and $10^{(n-1)}$ persons, where each car is owned by one person and each person owns 10 different cars, and each car has a color different from 'no-color', and each person has a name different from 'no-name'.
- We use a server machine, with Intel(R) Xeon(R) CPU E5-2620 v3 at 2.40GHz with 16 GB RAM, using MySQL 5.7.25.
- The execution-times (given in seconds) correspond to the arithmetic mean (average) of 50 executions.

- *How many cars have color 'no-color'?*

```
SELECT 1 AS val, COUNT(*) AS res
FROM
  (SELECT TEMP_gbody.val AS val, TEMP_gbody.ref_c AS res FROM
    (SELECT * FROM
      (SELECT CASE WHEN TEMP_LEFT.val = 0
        OR TEMP_RIGHT.val = 0 THEN 0 ELSE 1 END AS val,
        TEMP_LEFT.res = TEMP_RIGHT.res AS res,
        TEMP_LEFT.ref_c AS ref_c
      FROM
        (SELECT 1 AS val, color AS res, Car_id AS ref_c
          FROM Car
        )AS TEMP_LEFT JOIN
        (SELECT 1 AS val, 'no-color' AS res) AS TEMP_RIGHT
      ) AS TEMP_body
    WHERE TEMP_body.res = 1
  ) AS TEMP_gbody
) AS TEMP_src;
```

- *How many cars have color 'no-color'?*

```
SELECT COUNT(*)  
FROM  
  (SELECT *  
   FROM Car  
   WHERE color = 'no-color'  
  ) AS TEMP;
```

	CarDB(6) ¹	CarDB(7)
SQL	0.22	2.48
OCL2PSQL	0.24	3.00

¹The query generated by SQL-PL4OCL for a similar OCL expression takes 50.02 seconds to execute on an scenario like CarDB(6).

- *Is there at least one car whose color is different from 'no-color' ?*

► using COUNT:

```
SELECT COUNT(*) > 0 FROM
  (SELECT * FROM Car
   WHERE color <> 'no-color'
  ) AS TEMP;
```

► using EXISTS:

```
SELECT EXISTS
  (SELECT * FROM Car
   WHERE color <> 'no-color');
```

	CarDB(6)	CarDB(7)
SQL(using COUNT)	0.22	2.72
SQL(using EXISTS)	0.00	0.00
OCL2PSQL	0.24	3.10

- How many cars have at least one owner whose name is 'no-name' ?

► using correlated subquery:

```
SELECT COUNT(*)
FROM Car AS TEMP
WHERE EXISTS
  (SELECT 1
   FROM Ownership
   JOIN Person
   ON Person_id = owners
   WHERE name = 'no-name'
   AND TEMP.Car_id = ownedCars);
```

► using JOIN:

```
SELECT COUNT(*) FROM
  (SELECT 1 FROM Car
   JOIN Ownership
   ON Car_id = ownedCars
   JOIN Person
   ON Person_id = owners
   WHERE name = 'no-name'
   GROUP BY Car_id
  ) AS TEMP;
```

	CarDB(6)	CarDB(7)
SQL(using COUNT)	0.22	2.72
SQL(using EXISTS)	0.00	0.00
OCL2PSQL	0.24	3.10

Concluding remarks

- OCL plays a “starring” role in model-driven engineering. However, to fulfill this role, smart/advanced *code-generators* must be considered.
- Appropriate OCL-to-SQL *code-generators* can ease the work of database-centric developers, improve the final products quality and also reduce costs.
- OCL2PSQL is a novel mapping from a significant subset of OCL to “pure” SQL, taking into account the SQL *optimization engine*. It is an on-going project:
 - ▶ Rigorously extend the coverage in OCL, such as i) collection of collections, operations per se, generalizations, invalid values, etc.
 - ▶ Formally prove the correctness of the mapping, based on the semantics of SQL and OCL.